E.E.S.T 1 | 5°3° 2026

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

Participantes:
Martina Araujo, Thiago Goya y Sofia Salaberry

Escuela:
Escuela Técnica N°1 de Vicente Lopez

Curso:
Ciclo Superior 5°3°

Docentes a cargo:
Yamil Ganduglia, York Elias Mansilla Muñoz

Materia:
Lab. Programación

Año: 2026

Repositorio de GitHub:
Goya: https://github.com/GoyaYuki/Repaso_general

GitLab:
https://gitlab.com/GoyaYuki/Repaso_general

E.E.S.T 1 | 5°3° 2026

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

1. Introducción....3
2. Objetivos....3
3. reto 1: Recursividad....4
Explicación....4
Código....4
Funcionamiento....4
4. reto 2: Búsqueda Secuencial....4
Explicación....4
Explicación....4
Código....4
Funcionamiento....5
5. reto 3 : Intercambio con Punteros....5
Explicación....5
Código....6
Funcionamiento....6
8. CUESTIONARIO TÉCNICO DE CONTROL....6
Resolución....7
9. Evidencias....7
Figura 1:....7
Figura 3:....8
Figura 4:....8

E.E.S.T 1 | 5°3° 2026

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

1. Introducción

En esta actividad realizamos un repaso práctico de algunos de los conceptos de C++ relacionados con el funcionamiento de la memoria y el manejo de datos a bajo nivel. El objetivo fue comprender mejor cómo funciona la recursividad, la búsqueda secuencial en arreglos y los punteros mediante diferentes ejercicios prácticos.
Durante la actividad trabajamos con la pila de memoria o Stack, utilizando una función recursiva con un caso base para evitar que las llamadas continúen indefinidamente. También utilizamos un arreglo de datos para realizar una búsqueda secuencial y observar las direcciones de memoria de sus elementos.
Por último, trabajamos con punteros y pasaje por dirección para modificar directamente los valores originales de dos variables y realizar un intercambio entre ellas.

2. Objetivos

Los principales objetivos de esta actividad fueron:

- Comprender cómo funciona una función recursiva y la importancia del caso base.
- Comprender cómo se realiza una búsqueda secuencial dentro de un arreglo.
- Observar las direcciones de memoria de los elementos mediante el operador &.
- Comprender el funcionamiento de los punteros y la desreferenciación.
- Realizar un intercambio de valores utilizando punteros.
- Relacionar estos conceptos con el funcionamiento de la memoria a bajo nivel. Esto principalmente nos permitió poner en práctica lo solicitado por la actividad misma

3. reto 1: Recursividad

Explicación

- En el primer reto trabajamos con una función recursiva llamada sumaRecursiva(). Una función recursiva es aquella que se llama a sí misma para resolver un problema dividiéndolo en casos más pequeños.
  En este caso, utilizamos el número 5 y la función realiza la suma desde ese número hasta llegar a cero.

Código

- La parte del código que puede realizar esa función sería la siguiente

```
cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
int num = 5;
cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva
(num) << endl;
int sumaRecursiva(int n) {
    if (n <= 0) return 0; // Caso base obligatorio para prevenir Stack
    Overflow
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}
```

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

Funcionamiento

- En nuestro caso, la función recibe el número 5 y va restando 1 en cada llamada. Por eso comienza con sumaRecursiva(5) y después continúa con sumaRecursiva(4), sumaRecursiva(3) y así sucesivamente hasta llegar a sumaRecursiva(0).

4. reto 2: Búsqueda Secuencial

Explicación

- En el segundo reto trabajamos con un arreglo de números y una búsqueda secuencial. La búsqueda recorre los elementos uno por uno hasta encontrar el valor que estamos buscando. En este caso, buscamos el número 18 y mostramos su posición y dirección de memoria cuando lo encontramos.

Código

- La parte del código que puede realizar esa función sería la siguiente parte

```
cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
const int TAM = 10;
int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
int numeroBuscar = 18;
bool hallado = false;

for (int i = 0; i < TAM; i++) {
    if (vectorDatos[i] == numeroBuscar) {
        cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en
        el indice [" << i << "]" << endl;
        cout << "          Direccion RAM Hexadecimal: " << &
        vectorDatos[i] << endl;
        hallado = true;
        break; // Interrupción optimizada del ciclo O(n)
    }
}

if (!hallado) {
    cout << "[SISTEMA] El valor " << numeroBuscar << " no existe en
    el arreglo." << endl;
}
```

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

Funcionamiento

- En nuestro caso, el programa recorre el arreglo vectorDatos desde la posición [0], comparando cada valor con el número que estamos buscando.
  El número que buscamos es 18, y se encuentra en la posición [6] del arreglo.
  Cuando el programa encuentra el número, muestra su valor, su posición y su dirección de memoria.
  Además, utiliza break para detener el recorrido una vez que encuentra el número.
  De esta forma, no sigue recorriendo las posiciones que quedan después del elemento encontrado.

5. reto 3 : Intercambio con Punteros

Explicación

- En el tercer reto trabajamos con punteros para intercambiar los valores de dos variables. Los punteros permiten acceder directamente a la dirección de memoria de una variable y modificar el valor que se encuentra allí.
- En este caso, tenemos las variables x e y, con los valores 100 y 500. Pasamos sus direcciones a la función intercambiarValores() y, mediante los punteros, intercambiamos sus valores utilizando una variable temporal. De esta forma, al finalizar, x queda con 500 y y con 100.

Código

- La parte del código que puede realizar esa función sería la siguiente parte

```
    cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
    int x = 100, y = 500;
    cout << "Valores previos  -> X: " << x << " | Y: " << y << endl;

    // Invocación pasando direcciones de memoria
    intercambiarValores(&x, &y);

    cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;
    cout << "=====================================================" <<
    endl;

    return 0; // Código 0: Ejecución exitosa
}

// Desarrollo del intercambio físico mediante desreferenciación
void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}
```

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

Funcionamiento

- En nuestro caso, comenzamos con dos variables: x = 100 e y = 500.
  Luego, se pasan sus direcciones de memoria a los punteros ptrA y ptrB mediante &.
  Los punteros permiten acceder directamente a los valores de x e y usando *. Para realizar el intercambio se utiliza una variable temporal llamada temporal.
  Al finalizar, los valores quedan intercambiados: x = 500 e y = 100.

8. CUESTIONARIO TÉCNICO DE CONTROL

Investiguen y respondan estas 5 preguntas en su informe APA v7 en PDF para Moodle:

a. Caso Base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?
b. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?
c. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?
d. Variables Temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?
e. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

Resolución

a. La función recursiva continúa llamándose sin detenerse. Cada llamada agrega un marco a la pila de memoria (stack) hasta agotarla, lo que provoca un desbordamiento de pila (stack overflow) y puede hacer que el programa falle.
b. break mejora la eficiencia porque detiene el bucle apenas encuentra el elemento buscado. Así evita revisar las posiciones restantes del arreglo y reduce la cantidad de instrucciones que debe ejecutar el microprocesador.
c. Se puede utilizar el operador `&` para obtener y mostrar la dirección de memoria de cada elemento del arreglo. Al comparar las direcciones de los elementos consecutivos, se puede observar que se encuentran separadas por una cantidad constante de bytes. En el caso de un arreglo de `int`, normalmente la separación es de 4 bytes, aunque depende de la implementación.
d. La variable temporal permite guardar momentáneamente uno de los valores antes de reemplazarlo. Primero se guarda el valor de ptrA en temporal, después se copia el valor de ptrB en ptrA y finalmente se coloca el valor guardado en temporal dentro de ptrB. De esta manera se evita perder el valor original.
e. El reto que más me costó entender fue el intercambio de valores usando punteros, porque al principio no entendía bien la diferencia entre una variable y su dirección de memoria. Después entendí que & sirve para obtener la dirección de una variable y que * permite acceder al valor que está guardado en esa dirección. Al ver paso a paso cómo funcionaban ptrA, ptrB y la variable temporal, pude entender cómo se realiza el intercambio de los valores.

Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

9. Evidencias

Figura 1:

- Codigo ejecutable

```
PS C:\Users\Goyitas\Desktop\Repaso_general\src> cd "c:\Users\Goyitas\Desktop\Repaso_gen
eral\src\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
==================================================
  TALLER INTEGRADOR REPASO - ESTUDIANTE: Thiago Goya
==================================================

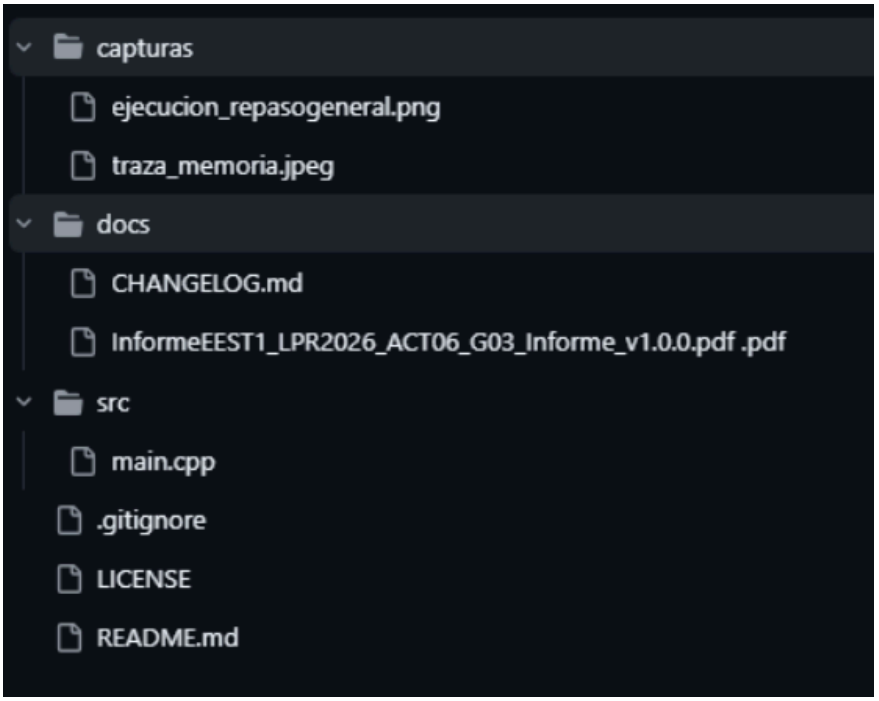
=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===
[SISTEMA] Valor 18 hallado en el indice [6]
          Direccion RAM Hexadecimal: 0x790b7ffc58

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos  -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
==================================================
```

Figura 2:

- Estructura del Repositorio



Actividad N6: Repaso General

Martina Araujo, Thiago Goya y Sofia Salaberry

Figura 3:

- Ejecución del comando git remote -v

```
PS C:\Users\Goyitas\Desktop\Repaso_general> git remote -v
origin  https://github.com/GoyaYuki/Repaso_general.git (fetch)
origin  https://github.com/GoyaYuki/Repaso_general.git (push)
origin  https://gitlab.com/GoyaYuki/Repaso_general.git (push)
```

Figura 4:

- Diagrama de ram

